C++71 期专写 BUG 组本周学习情况总结

组长：郭子豪

组员：张建霖、张祖成、顾李恒、蔡家盛、许奎泽

时间：2026.3.23— 2026.3.28

会议主讲人：郭子豪

会议纪要人：郭子豪

参会人：张建霖、张祖成、顾李恒、蔡家盛、许奎泽、李旺、余文嘉、刘真玮

请假人及未到原因：无

1、上课打卡情况：

姓 名	星期一	星期二	星期三	星期四	星期五
郭子豪					
张建霖					
张祖成					
顾李恒					
蔡家盛					
许奎泽					

2、作业完成情况：截至 2026-3-28

姓 名	day29	day30	day31	day32	day33
郭子豪					
张建霖					
张祖成					
顾李恒					
蔡家盛					

许奎泽					

3、组内分享内容摘要：

一、核心问题与方法概述

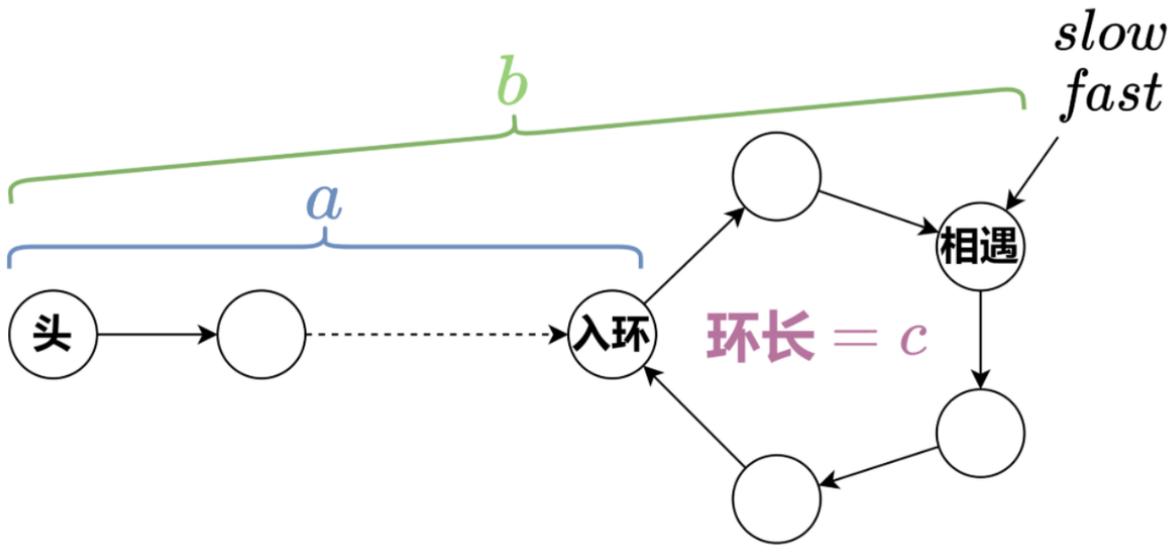
本次组会聚焦**链表中环的检测**及**入环口定位**问题，采用经典的**快慢指针（Floyd 判圈算法）** 进行分析与实现。

1. 环的存在性判断

- **核心思路**：使用两个指针同时遍历链表，慢指针（slow）每次前进 1 步，快指针（fast）每次前进 2 步。
- **判断逻辑**：
  - 若链表存在环：快慢指针最终必然在环内相遇；
  - 若链表无环：快指针会先到达链表末尾，二者永不相遇。
- **本质**：快指针相对慢指针的速度为 1 步 / 次，在环中必然会追上慢指针。

2. 入环口定位的数学推导

在确认链表存在环后，我们通过以下推导定位入环口：



- 定义变量：
  - 头节点到入环口的步数：a
  - 环的总长度：c
  - 相遇时慢指针走过的步数：b，则快指针走过 2b 步
- 关键等式：快指针比慢指针多走了 k 圈环，因此：2b−b=kc b=kc

- 推导结论：慢指针从入环口到相遇点走了 $b-a=kc-a$ 步，这意味着：**从相遇点再走 a 步，恰好回到入环口。**
- 实操方法：将慢指针留在相遇点，让头节点指针和慢指针**同时以 1 步 / 次前进**，二者相遇的节点即为**入环口。**

二、算法实现要点

1. **边界处理**：
   a. 空链表或单节点链表直接判定无环；
   b. 遍历过程中需判断快指针是否到达链表尾部（fast == null 或 fast.next == null）。
2. **代码实现**：

```
struct ListNode* detectCycle(struct ListNode* head) {
    struct ListNode* slow = head;
    struct ListNode* fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (fast == slow) { // 相遇
            while (slow != head) { // 再走 a 步
                slow = slow->next;
                head = head->next;
            }
            return slow;
        }
    }
    return NULL;
}
```

3. **时间与空间复杂度**：
   a. 时间复杂度：$O(n)$，仅需两次线性遍历；
   b. 空间复杂度：$O(1)$，仅使用常数级额外指针，无额外数据结构。

三、讨论与拓展

1. **与哈希表法对比**：
   a. 哈希表法：遍历同时记录节点地址，首次重复节点即为入环口，实现简单但空间复杂度为 O(n)；
   b. 快慢指针法：空间复杂度更优（O(1)），适合内存受限场景。
2. **特殊场景验证**：
   a. 环的长度为 1（自环）：算法依然有效；
   b. 头节点即为入环口：推导与代码逻辑同样成立。
3. **应用场景**：
   a. 检测内存泄漏；
   b. 图论中检测有向图的环；
   c. 循环队列、循环缓冲区的边界校验。

4、课后疑问

(1)

(2)

(3)

5、其他意见或建议：

(1)

(2)

(3)

6、组会总结（2026.3.28）：

本次组会通过**数学推导 + 算法实现**的方式，系统梳理了快慢指针法在链表环问题中的应用：

- 掌握了环存在性的高效判断方法；
- 理解了入环口定位的核心数学原理；
- 对比了不同算法的优劣，明确了适用场景。

关联题目：LeetCode141、142 题